

CerbaSeal

Core · v0.1.0

System Architecture & Technical Reference Manual

The definitive technical reference for CerbaSeal-Core — a deterministic enforcement gate that makes it technically impossible to execute a consequential AI-assisted action without satisfying every governance requirement. Covers invariant framework, domain type system, audit chain, HTTP API, deployment architecture, and partner delivery.

VERSION	STATUS	DATE	AUTHOR
0.1.0	GENERALLY AVAILABLE	June 2026	Jesse Lamont

Contents

1	Executive Summary	3
2	System Overview	4
3	Core Architecture	5
3.1	Enforcement Gate — ExecutionGateService	5
3.2	Gate-Issued Result Registry	6
3.3	Decision Flow	6
4	Invariant Framework	7
5	Domain Type Reference	9
5.1	GovernedRequest	9
5.2	GateResult	11
5.3	DecisionEnvelope	11
5.4	ReleaseAuthorization	12
5.5	BlockedActionRecord	12
5.6	Supporting Types	13
6	Reason Code Reference	14
7	Configuration System	15
7.1	CerbaSealConfig — Authority, Workflow & Action Classes	15
7.2	CerbaSealPolicy — Actor Mappings, Approval Chains, Rules	16
8	Audit & Evidence Chain	17
8.1	AppendOnlyLogService (In-Memory)	17
8.2	FileBackedAppendOnlyLogService (Persistent)	17
8.3	EvidenceBundleService	18
9	HTTP API Reference	18
10	Deployment Architecture	19
10.1	Path A — Docker Compose (Recommended)	19
10.2	Path B — Node.js Direct	20
11	Test Coverage	21
12	Partner Delivery Framework	22
13	Security Properties & Threat Model	24

1

Executive Summary

CerberaSeal-Core is a deterministic execution governance library for organisations that use AI-assisted decision making in regulated workflows. It solves a structural problem that exists in virtually every AI integration: the path from "AI recommends" to "action executes" is unenforced. Any sufficiently motivated code path can bypass the recommendation, skip approvals, omit logging, or act on a stale AI output. CerberaSeal makes that bypass technically impossible.

The enforcement gate evaluates every governed request against 12 hard invariants before issuing a release authorisation. The invariants are unconditional — no configuration, no policy, no caller can relax them. A client-authored policy layer adds client-specific restrictions on top. The net effect is a system where the only path to a release authorisation is a request that satisfies every invariant and every policy rule.

Key Guarantees

Single point of control:

Every consequential action must pass through Execution GateService.evaluate(). There is no secondary API, no override, no bypass.

Deterministic decisions:

The same request, evaluated twice, always produces the same decision. The gate contains no randomness, no network calls, no side effects.

Immutable evidence: Every decision produces a DecisionEnvelope marked immutable: true. The evidence bundle captures the complete decision context and cannot be modified.

Tamper-evident audit log:

The audit chain uses SHA-256 forward chaining. Any modification to a historical entry breaks the chain, and verifyChain() detects it immediately.

AI non-authoritativeness:

An actor with authority class "ai" is structurally incapable of authorising its own proposals. INV-05 is checked unconditionally on every request.

Policy cannot relax

invariants: The policy layer is additive. It can impose

additional restrictions. It cannot grant exemptions to the 12 core invariants.

Scope of v0.1.0

v0.1.0 covers the Drop 01 scope: the enforcement gate, audit chain, evidence layer, configuration system, policy layer, HTTP deployment server, and partner delivery kit. Cryptographic signing (HMAC over DecisionEnvelope) is available in the source but not yet exposed as a first-class API. Third-party security review is planned for v0.2.0.

What CerbaSeal Is Not

- CerbaSeal is not an AI model, a workflow engine, or a case management system.
 - It does not make decisions — it evaluates whether a proposed decision is authorised to execute.
 - It does not replace compliance tooling — it provides the enforcement foundation that compliance tooling audits.
 - It is not a hosted service — it is a library deployed inside the client's own infrastructure.
-

2 System Overview

The 7-Layer Architecture

CerbaSeal is structured as 7 layers, each with a discrete responsibility. The request flows upward through the stack; the evidence flows downward and is captured at every layer.

System architecture — 7 layers

[illegible]

Layer Responsibilities

Layer	Component	Responsibility	Partner Touch-Point
1 — Request	GovernedRequest	Structured input: actor, action, provenance, trust state	Client integration constructs the request object
2 — Trust	TrustState	Assert actor identity is currently trusted	Client IdP populates trusted: true/false
3 — Provenance	ProvenanceRef	Trace AI model version, rule set, source hash	Client AI pipeline exposes model metadata
4 — Gate	ExecutionGateService	Evaluate 12 invariants + policy. Issue GateResult.	Partners instantiate; never modify gate source
5 — Policy	CerbaSealPolicy	Add client-specific approval chains and role mappings	Partners author cerbaseal.policy.json per client
6 — Audit	AppendOnlyLogService	Append-only JSONL log with SHA-256 forward chain	Partners configure log path and retention
7 — Evidence	EvidenceBundleService	Assemble full proof bundle; drive ExportManifest	Partners run export:proof at pilot close

Decision Lifecycle

A governed request passes through the following stages before any action may execute:

Construct: Client builds a GovernedRequest from its AI system output, identity context, and control state.

Submit: GovernedRequest is passed to gate.evaluate(). This is a synchronous, pure

function call.

Invariant check: The gate evaluates all 12 invariants in sequence. The first failure produces a REJECT or HOLD.

Policy check: If invariants pass, policy rules (approval chains, action policies, workflow rules) are evaluated.

Decision: A DecisionEnvelope is produced with finalState ALLOW, HOLD, or REJECT. It is marked immutable.

Release: For ALLOW: a ReleaseAuthorization is issued. For HOLD/REJECT: a BlockedActionRecord is issued.

Audit: The calling code appends the outcome to the audit log. The chain hash is updated.

Evidence: EvidenceBundleService assembles the full bundle; export:proof generates the evidence package.

3

Core Architecture

3.1 Enforcement Gate — ExecutionGateService

ExecutionGateService is the single enforcement point. It is instantiated once per process, typically at application startup, with the client config and policy loaded from disk. Its evaluate() method is the only public mutation-bearing operation.

Instantiation and evaluation

```
import { ExecutionGateService } from "cerbaseal-review/src/services/execution/execution-gate-service.js";
import { loadCerbaSealConfig } from "cerbaseal-review/src/config/cerbaseal-config.js";
import { loadCerbaSealPolicy } from "cerbaseal-review/src/config/cerbaseal-policy.js";

const config = loadCerbaSealConfig(); // reads cerbaseal.config.json
const policy = loadCerbaSealPolicy(); // reads cerbaseal.policy.json
const gate = new ExecutionGateService(config, policy);

const result = gate.evaluate(request); // returns GateResult
// result.decisionEnvelope.finalState != "ALLOW" | "HOLD" | "REJECT"
```

Constructor

Parameter	Type	Required	Purpose
config	CerbaSealConfig	No (default: {})	Extended authority, workflow, and action class registrations
policy	CerbaSealPolicy	No (default: undefined)	Actor mappings, approval chains, workflow rules, action policies

evaluate(request: GovernedRequest) != GateResult

evaluate() is a synchronous, deterministic, side-effect-free function. It throws CerbaSealError for malformed input. It never resolves asynchronously. The return value is a GateResult registered in the internal WeakSet (see §3.2).

Invariant evaluation order within evaluate():

Invariant evaluation sequence (in order)

INV-11	REQUEST_SCHEMA_AND_ACTION_CLASS_VALID	– Schema and action class validation (first; fast fail)
INV-12	PROPOSAL_AND_REQUEST_ACTION_MUST_MATCH	– Proposed action matches request action
INV-01	NO_POLICY_PACK_NO_EXECUTION	– Policy pack reference present
INV-02	NO_PROVENANCE_NO_ACTION	– Provenance reference present
INV-04	NO_LOGGING_NO_EXECUTION	– Logging ready flag asserted
INV-09	TRUST_STATE_REQUIRED	– Trust state present and trusted
INV-10	PROHIBITED_USE_MUST_BLOCK	– prohibitedUse flag not set
INV-05	AI_NON_AUTHORITATIVE	– Actor authority class is not "ai"
INV-08	STALE_CONTROLS_BLOCK_SENSITIVE_RELEASE	– Control status valid for sensitive requests
INV-03	NO_REQUIRED_APPROVAL_NO_RELEASE	– Approval present when required
INV-06	NO_BYPASS_OF_EXECUTION_GATE	– Result registered in gate-issued WeakSet
INV-07	IMMUTABLE_DECISION_ENVELOPE	– Envelope marked immutable: true

3.2 Gate-Issued Result Registry

A module-private WeakSet (`_gateIssuedResults`) tracks every `GateResult` returned by `evaluate()`. `EvidenceBundleService` calls `assertIsGateIssued()` before assembling an evidence bundle. Any `GateResult` that was not produced by `evaluate()` — including hand-constructed objects — triggers an immediate `REJECT`. This closes the path where a caller self-constructs a `GateResult` to bypass the evidence layer.

Security note: The WeakSet is private to the `execution-gate-service` module. It cannot be accessed or modified from outside the module boundary. There is no API to register a `GateResult` externally.

3.3 Decision Flow

Decision tree

```
gate.evaluate(request)
%
% % INV-11,12 fails !' throw CerbaSealError(REJECT)
%
% % INV-01,02,04,09,10 fails !' GateResult{ finalState: "REJECT", releaseAuth: null, blocked:
record }
%
% % INV-05 fails (AI actor) !' GateResult{ finalState: "REJECT", blocked: record }
%
% % INV-08 fails (stale controls, sensitive) !' GateResult{ finalState: "REJECT" }
%
% % INV-03 fails (approval required, not present) !' GateResult{ finalState: "HOLD" }
%   % % approval present but authority class invalid !' GateResult{ finalState: "REJECT" }
%
% % All invariants pass !' Policy checks
% %   workflowRules: requiresApproval true and approval absent !' HOLD
% %   actionPolicies: blocked !' REJECT
% %   actionPolicies: requires_approval and absent !' HOLD
% %   All pass !' GateResult{ finalState: "ALLOW", releaseAuth: ReleaseAuthorization }
```


4

Invariant Framework

The 12 invariants are unconditional enforcement rules. They are not configurable, overridable, or bypassable via policy. Every evaluated request is checked against all applicable invariants. A failure at any invariant halts further evaluation and produces a terminal REJECT or a HOLD decision.

Invariant contract: A client can never configure their way around a core invariant. The policy layer can only add restrictions — never remove them. This is the foundational guarantee that CerbaSeal provides to compliance teams and auditors.

Code	Name	What It Enforces	Failure State
INV-01	NO_POLICY_PACK_NO_EXECUTION	policyPackRef must be non-null on every request. No policy reference means no execution path exists.	REJECT
INV-02	NO_PROVENANCE_NO_ACTION	provenanceRef must be non-null. Every action must be traceable to the AI model and rule set that generated it.	REJECT
INV-03	NO_REQUIRED_APPROVAL_NO_RELEASE	When approvalRequired is true (or workflow policy requires it), a valid ApprovalArtifact must be present and its authority class accepted.	HOLD / REJECT
INV-04	NO_LOGGING_NO_EXECUTION	loggingReady must be true. The caller asserts that an audit log is ready to receive the outcome before the gate issues a release.	REJECT
INV-05	AI_NON_AUTHORITATIVE	An actor with actorAuthorityClass "ai" cannot authorise its own proposals. AI is proposal-only; a human authority class must provide approval.	REJECT
INV-06	NO_BYPASS_OF_EXECUTION_GATE	GateResult objects used to assemble evidence must have been produced by gate.evaluate(). Hand-constructed results are rejected by assertIsGateIssued().	REJECT
INV-07	IMMUTABLE_DECISION_ENVELOPE	The DecisionEnvelope is marked immutable: true at issuance. The type system and WeakSet registration enforce that no mutation can occur post-issuance.	REJECT
INV-08	STALE_CONTROLS_BLOCK_SENSITIVE_RELEASE	For requests with sensitive: true, controlStatus.criticalControlsValid must be true and controlStatus.stale must be false.	REJECT
INV-09	TRUST_STATE_REQUIRE_D	trustState.trusted must be true. A request from an untrusted actor (expired session, revoked token) is rejected unconditionally.	REJECT
INV-10	PROHIBITED_USE_MUST_BLOCK	prohibitedUse: true triggers an immediate REJECT regardless of any other field. No policy override can unblock a prohibited use.	REJECT
INV-11	REQUEST_SCHEMA_AND_ACTION_CLASS_VALID	proposedActionClass must be one of the registered action classes (core or extended). An unknown action class produces a CerbaSealError throw.	REJECT (throw)
INV-12	PROPOSAL_AND_REQUEST_ACTION_MUST_MATCH	proposal.requestedActionClass must equal proposedActionClass. A proposal for a different action than the request declares is structurally invalid.	REJECT

5

Domain Type Reference

5.1 GovernedRequest

GovernedRequest is the single input to `gate.evaluate()`. Every field carries a specific governance meaning. No field is purely informational — each is evaluated by at least one invariant or policy rule.

Field	Type	Required	Invariant / Use
requestId	string	Yes	Unique request identifier. Used to correlate audit log entries and evidence bundles.
workflowClass	"fraud_triage" "transaction_escalation"	Yes	Identifies the workflow context. Drives approval chain selection and action policy lookup.
jurisdiction	string	Yes	Jurisdiction identifier (e.g. "US-CA", "EU-GDPR"). Informational for evidence; available for policy
actorId	string	Yes	Identifier of the actor submitting the request. Included in evidence bundle and audit log.
actorAuthorityClass	AuthorityClass	Yes	INV-05: must not be "ai". Determines approval authority eligibility.
proposedActionClasses	UnknownableActionClass	Yes	INV-11: must be a registered action class. INV-12: must match proposal.requestedActionClass.
proposal	DecisionProposal	Yes	The AI or rule-engine proposal. proposalSourceKind, confidence, requestedActionClass, reasonCodes.
sensitive	boolean	Yes	INV-08: if true, controlStatus must be valid and non-stale.
prohibitedUse	boolean	Yes	INV-10: if true, immediate REJECT regardless of all other fields.
policyPackRef	PolicyPackRef null	Yes	INV-01: must be non-null. { id: string, version: string }.
provenanceRef	ProvenanceRef null	Yes	INV-02: must be non-null. { modelVersion, ruleSetVersion, sourceHash }.
approvalRequired	boolean	Yes	INV-03: if true, approvalArtifact must be present and valid.
approvalArtifact	ApprovalArtifact null	Yes	INV-03: must be present when approvalRequired true. Contains approverId, authority class, signature.
loggingReady	boolean	Yes	INV-04: must be true. Caller asserts audit log is ready.
controlStatus	ControlStatus	Yes	INV-08: { criticalControlsValid, stale, verificationRunId }.
trustState	TrustState	Yes	INV-09: { trusted: boolean, trustStateId: string }. trusted must be true.
createdAt	string (ISO 8601)	Yes	Request creation timestamp. Included in audit log and evidence bundle.

GovernedRequest — Supporting Types

Type	Fields	Notes
PolicyPackRef	{ id: string, version: string }	Identifies the policy pack applied to this request. Must match the deployed cerbaseal.policy.json.
ProvenanceRef	{ modelVersion: string, ruleSetVersion: string, sourceHash: string }	Traces the AI model and rule set that produced the proposal. All three fields required.
DecisionProposal	{ proposalSourceKind, authorityBearing, requestedActionClass, confidence, reasonCodes, proposalCreatedAt }	proposalSourceKind: "ai" "deterministic_rule". authorityBearing must be false for AI actors (enforced by INV-05).
ApprovalArtifact	{ approvalId, approverId, forRequestId, approverAuthorityClass, privilegedAuthSatisfied, immutableSignature, approvedAt }	approverAuthorityClass must be a HumanAuthorityClass (analyst reviewer manager compliance_officer). immutableSignature is an opaque string.
ControlStatus	{ criticalControlsValid: boolean, stale: boolean, verificationRunId: string }	Evaluated against sensitive requests (INV-08). verificationRunId provides audit traceability.
TrustState	{ trusted: boolean, trustStateId: string }	Evaluated by INV-09. trustStateId provides audit traceability. A single false trusted flag unconditionally rejects.

5.2 GateResult

GateResult is the return value of `gate.evaluate()`. It is a value object registered in the internal WeakSet at the point of issuance. All three fields are always present; `releaseAuthorization` and `blockedActionRecord` are mutually exclusive with `finalState`.

GateResult interface

```
interface GateResult {
  decisionEnvelope: DecisionEnvelope;           // always present; immutable: true
  releaseAuthorization: ReleaseAuthorization | null; // non-null iff ALLOW
  blockedActionRecord: BlockedActionRecord | null; // non-null iff HOLD or REJECT
}
```

5.3 DecisionEnvelope

DecisionEnvelope is the primary artifact of a gate evaluation. It is the object that should be persisted, logged, and referenced in compliance records. It carries the full decision trace including which invariants were checked and which reason codes were applied.

Field	Type	Notes
envelopeld	string	Deterministic: "env_" + requestId. Stable identifier for this decision.
requestId	string	Echoes the GovernedRequest requestId. Correlation key.
workflowClass	WorkflowClass	Echoes the workflow class from the request.
finalState	"ALLOW" "HOLD" "REJECT"	The gate decision. Terminal.
permittedActionClass	ActionClass null	Non-null iff finalState is ALLOW. The specific action that may be executed.
humanApprovalRequired	boolean	True when an approval was required by invariant or policy.
humanApprovalPresent	boolean	True when a valid ApprovalArtifact was provided and accepted.
proposalSourceKind	"ai" "deterministic_rule"	Source of the proposal. Copied from the request for evidence.
immutable	true	Literal type true. Enforced at both TypeScript and runtime level.
evidenceBundleId	string	Deterministic: "evidence_" + requestId. Cross-references the evidence bundle.
trace	DecisionTrace	{ checkedInvariants: InvariantCode[], reasonCodes: ReasonCode[], evaluatedAt: string }
issuedAt	string (ISO)	Timestamp of gate evaluation.

5.4 ReleaseAuthorization

ReleaseAuthorization is issued only when finalState is ALLOW. It constitutes the formal permission for the proposed action to execute. It must be presented to the action executor to confirm the gate approved the execution.

ReleaseAuthorization interface

```
interface ReleaseAuthorization {
  releaseAuthorizationId: string; // "release_" + requestId
  requestId: string;
  envelopeId: string;
  actionClass: ActionClass;
  releasedAt: string; // ISO timestamp
}
```

5.5 BlockedActionRecord

BlockedActionRecord is issued for HOLD and REJECT outcomes. It records the specific reason codes and invariant checks that produced the blocking decision. It is the compliance record for a failed or held governance event.

BlockedActionRecord interface

```
interface BlockedActionRecord {
  requestId: string;
  finalState: "HOLD" | "REJECT";
  reasonCodes: ReasonCode[];
  checkedInvariants: InvariantCode[];
  recordedAt: string; // ISO timestamp
}
```

5.6 Supporting Types — AuthorityClass & WorkflowClass

Type	Values	Notes
AuthorityClass	"system" "ai" "analyst" "reviewer" "manager" "compliance_officer"	Core set. "ai" cannot be an approver (INV-05). Additional classes registered via CerbaSealConfig
HumanAuthorityClasses	"analyst" "reviewer" "manager" "compliance_officer"	Subset of AuthorityClass that may appear as approverAuthorityClass in ApprovalArtifact
WorkflowClass	"fraud_triage" "transaction_escalation" "account_hold_recommendation"	Core set. "fraud_triage" always requires approval (WORKFLOWS_REQUIRING_APPROVAL). Extended via config
ActionClass	"allow" "hold" "reject" "escalate" "account_hold"	Core set. Extended via CerbaSealConfig.actionClasses.extended.
DecisionState	"ALLOW" "HOLD" "REJECT"	The three terminal states of a gate evaluation.
ProposalSourceKind	"ai" "deterministic_rule"	Source of the action proposal. AI proposals require human approval for sensitive actions.

6

Reason Code Reference

Reason codes appear in `DecisionTrace.reasonCodes` and `BlockedActionRecord.reasonCodes`. They identify the specific enforcement rule or condition that produced the decision. Multiple reason codes may be present in a single decision trace.

Reason Code	Category	Meaning
DECISION_ALLOWED	Terminal — Allow	All invariants and policy rules passed. Release authorisation issued.
DECISION_HELD	Terminal — Hold	Request is valid but requires human approval before release.
DECISION_REJECTED	Terminal — Reject	One or more invariants failed. Request blocked, no release path.
NO_POLICY_PACK	Invariant (INV-01)	<code>policyPackRef</code> is null. No policy reference means no execution path.
NO_PROVENANCE	Invariant (INV-02)	<code>provenanceRef</code> is null. Proposal is not traceable to a model or rule set.
REQUIRED_APPROVAL_MISSING	Invariant (INV-03)	<code>approvalRequired</code> true but <code>approvalArtifact</code> is null.
INVALID_APPROVAL_AUTHORITY	Invariant (INV-03)	Approval present but <code>approverAuthorityClass</code> not accepted by the approval chain.
PRIVILEGED_AUTH_NOT_SATISFIED	Invariant (INV-03)	Approval present but <code>privilegedAuthSatisfied</code> is false.
APPROVAL_SIGNATURE_MISSING	Invariant (INV-03)	<code>ApprovalArtifact</code> present but <code>immutableSignature</code> is absent or empty.
LOGGING_NOT_READY	Invariant (INV-04)	<code>loggingReady</code> is false. Caller has not confirmed audit log readiness.
AI_CANNOT_AUTHORIZE	Invariant (INV-05)	<code>actorAuthorityClass</code> is "ai". AI actors cannot be the approving authority.
PROHIBITED_USE	Invariant (INV-10)	<code>prohibitedUse</code> is true. Unconditional block; no policy can override.
CONTROL_STALE_OR_INVALID	Invariant (INV-08)	Sensitive request and <code>controlStatus.stale</code> is true or <code>criticalControlsValid</code> is false.
TRUST_STATE_INVALID	Invariant (INV-09)	<code>trustState.trusted</code> is false. Actor session or token is not trusted.
UNKNOWN_ACTION_CLASS	Invariant (INV-11)	<code>proposedActionClass</code> is not in the registered action class set.
MALFORMED_REQUEST	Schema	Request object fails schema validation. Also used for internal gate-bypass detection.
INVALID_PROPOSAL	Invariant (INV-12)	<code>proposal.requestedActionClass</code> does not match <code>proposedActionClass</code> .

7

Configuration System

7.1 CerbaSealConfig — Class Extension Registry

CerbaSealConfig extends the enforcement gate with client-specific authority, workflow, and action class identifiers. The core sets are hardcoded in the gate source and cannot be removed. Extended classes are additive — they register new identifiers that the gate will accept on requests.

CerbaSealConfig interface

```
interface CerbaSealConfig {
  authorityClasses?: CerbaSealClassExtension; // adds new actor authority levels
  workflowClasses?: CerbaSealClassExtension; // adds new workflow identifiers
  actionClasses?: CerbaSealClassExtension; // adds new action class values
}

interface CerbaSealClassExtension {
  core?: readonly string[]; // documentation only; gate ignores at runtime
  extended: readonly string[]; // classes the gate will accept in requests
}
```

Core classes (always registered, cannot be removed):

Type	Core Values
authorityClasses	"system", "ai", "analyst", "reviewer", "manager", "compliance_officer"
workflowClasses	"fraud_triage" (always requires approval), "transaction_escalation", "account_hold_recommendation"
actionClasses	"allow", "hold", "reject", "escalate", "account_hold"

cerbaseal.config.json example

```
// Example cerbaseal.config.json
{
  "authorityClasses": {
    "core": ["system", "ai", "analyst", "reviewer", "manager", "compliance_officer"],
    "extended": ["risk_officer", "senior_analyst", "director"]
  },
  "workflowClasses": {
    "core": ["fraud_triage", "transaction_escalation", "account_hold_recommendation"],
    "extended": ["insurance_claim_review", "kyc_escalation"]
  }
}
```

7.2 CerbaSealPolicy — Client-Specific Rules

CerbaSealPolicy carries four categories of client-specific governance rules. Policy rules are evaluated after all 12 invariants pass. They can only add restrictions; they cannot grant exemptions to invariants.

CerbaSealPolicy interface

```
interface CerbaSealPolicy {
  actorMappings?: Record<string, AuthorityClass>; // role name != authority class
  approvalChains?: Record<string, AuthorityClass[]>; // workflow != [approved authority classes]
  workflowRules?: WorkflowRule[]; // { workflowClass, requiresApproval: boolean }
  actionPolicies?: Record<string, Record<string, PolicyActionBehavior>>; // workflow != action !=
  behaviour
}

type PolicyActionBehavior = "requires_approval" | "auto_allow" | "blocked";
```

Field	Purpose	Example
actorMappings	Translates client role name strings to CerbaSeal authority classes. Allows clients to pass their internal role names in requests without TypeScript	{ "Head of Risk": "manager", "Senior Fraud Analyst": "analyst" }
approvalChains	Per-workflow list of authority classes whose approvals the gate will accept. If absent for a workflow, default authority class logic applies.	{ "fraud_triage": ["manager","compliance_officer"] }
workflowRules	Explicitly toggles approval requirement for a workflow class. Overrides caller-supplied approvalRequired for matched workflows.	{ workflowClass: "kyc_escalation", requiresApproval: true }
actionPolicies	Per-workflow, per-action behaviour override. "blocked" causes REJECT. "requires_approval" causes HOLD if approval absent. "auto_allow" permits without additional constraints.	{ "fraud_triage": { "allow": "requires_approval" } }

8

Audit & Evidence Chain

8.1 AppendOnlyLogService (In-Memory)

AppendOnlyLogService is the in-memory audit log implementation. Each appended event is hashed with SHA-256 and chained to the previous entry. The chain is verifiable at any time via `verifyChain()`, which traverses all entries and validates every hash link.

IAuditLogService and AuditLogEntry interfaces

```
interface IAuditLogService {
    append(event: AuditEventPayload): AuditLogEntry;
    list(): AuditLogEntry[];
    listByRequestId(requestId: string): AuditLogEntry[];
    verifyChain(): boolean; // returns true iff every hash link is valid
}

interface AuditLogEntry {
    eventId:      string;      // UUID
    requestId:    string;
    eventType:    AuditEventType; // "REQUEST_EVALUATED" | "RELEASE_AUTHORIZED" | ...
    timestamp:    string;
    payloadHash:  string;      // SHA-256 of the event payload
    previousHash: string | null; // null for the first entry
    entryHash:    string;      // SHA-256 of (payloadHash + previousHash + metadata)
}
```

8.2 FileBackedAppendOnlyLogService (Persistent)

FileBackedAppendOnlyLogService extends the in-memory implementation with JSONL persistence. Each `append()` call writes synchronously to disk before returning (using `appendFileSync`). The chain is loaded and verified from disk on construction — if the chain is broken on disk, `verifyChain()` returns false immediately.

Storage format: JSONL —

one JSON-serialised

AuditLogEntry per line

Write durability:

`appendFileSync` — crash-

safe; the entry is on disk

before `append()` returns

Chain loading: On

construction, all entries are

read from disk and the chain

is re-constituted in memory

Verification: `verifyChain()`

runs the full SHA-256 chain

walk over the in-memory copy

Mutation safety: `deepClone`

applied to all returned values

— callers cannot mutate
internal state

Production deployment note: Set LOG_PATH to a persistent volume path (e.g. /app/data/audit.jsonl). The Docker Compose configuration mounts ./data:/app/data to provide this persistence across container restarts.

8.3 EvidenceBundleService

EvidenceBundleService assembles an EvidenceBundle from a GovernedRequest and a GateResult. It first calls assertIsGateIssued() to verify the GateResult was produced by the gate (not hand-constructed). It then appends multiple audit events and assembles the complete bundle snapshot.

Audit Event	AuditEventType	When Appended
Request evaluated	"REQUEST_EVALUATED"	Always — first event in the bundle
Release authorised	"RELEASE_AUTHORIZED"	When finalState is ALLOW
Action blocked	"ACTION_BLOCKED"	When finalState is HOLD or REJECT
Evidence bundle created	"EVIDENCE_BUNDLE_CREATED"	After bundle assembly
Export manifest created	"EXPORT_MANIFEST_CREATED"	When export:proof is run

The EvidenceBundle object contains: evidenceBundleId, the original GovernedRequest, the DecisionEnvelope, the ReleaseAuthorization or null, the BlockedActionRecord or null, the full eventChain (all AuditLogEntry objects for this requestId), and a createdAt timestamp.

9

HTTP API Reference

The deployment-starter/index.ts HTTP server exposes the enforcement gate as a minimal REST API. It is the integration surface for clients that cannot embed the TypeScript library directly (e.g. Python or Go services). Both endpoints are unauthenticated — network-level access control is the responsibility of the operator.

POST /evaluate

Property	Detail
Method + Path	POST /evaluate
Content-Type	application/json
Request body	A JSON-serialised GovernedRequest object (see §5.1 for full field reference)
Success response	200 OK — JSON-serialised GateResult { decisionEnvelope, releaseAuthorization, blockedActionRecord }
Client error	400 Bad Request — { "error": "Invalid JSON — expected a GovernedRequest object" }
Gate error	422 Unprocessable Entity — { "error": "<CerbaSealError message>" }
Side effect	On success: appends a REQUEST_EVALUATED event to the audit log service

curl example — ALLOW scenario

```
# Submit a governed request
curl -X POST http://localhost:3000/evaluate \
  -H "Content-Type: application/json" \
  -d '{
    "requestId": "req-001",
    "workflowClass": "fraud_triage",
    "jurisdiction": "US-CA",
    "actorId": "analyst-jane",
    "actorAuthorityClass": "analyst",
    "proposedActionClass": "allow",
    "proposal": {
      "proposalSourceKind": "ai",
      "authorityBearing": false,
      "requestedActionClass": "allow",
      "confidence": 0.94,
      "reasonCodes": ["MODEL_HIGH_CONF"],
      "proposalCreatedAt": "2026-06-08T18:00:00Z"
    },
    "sensitive": true,
    "prohibitedUse": false,
    "policyPackRef": { "id": "client-policy", "version": "1.0.0" },
    "provenanceRef": { "modelVersion": "v3.1", "ruleSetVersion": "v2.0", "sourceHash":
"sha256:abc123" },
    "approvalRequired": true,
    "approvalArtifact": {
      "approvalId": "appr-001", "approverId": "mgr-john", "forRequestId": "req-001",
      "approverAuthorityClass": "manager", "privilegedAuthSatisfied": true,
      "immutableSignature": "sig-abc", "approvedAt": "2026-06-08T18:00:10Z"
    },
    "loggingReady": true,
    "controlStatus": { "criticalControlsValid": true, "stale": false, "verificationRunId":
"run-001" },
    "trustState": { "trusted": true, "trustStateId": "ts-001" },
    "createdAt": "2026-06-08T18:00:00Z"
  }'
```

GET /health

Property	Detail
Method + Path	GET /health
Purpose	Gate readiness check. Used by Docker Compose HEALTHCHECK directive and upstream load balancers.
Response	200 OK always (including "degraded" state — caller interprets status field)
Response body	{ "status": "ok" "degraded", "gateReady": true, "auditChainValid": boolean, "uptime": number (seconds) }
Degraded state	auditChainValid: false — the audit log chain has been broken. Indicates potential tampering or disk corruption.
gateReady	Always true if the server started cleanly. The gate service loaded config and policy at startup.

Healthy response example

```
$ curl http://localhost:3000/health
{
  "status": "ok",
  "gateReady": true,
  "auditChainValid": true,
  "uptime": 142.7
}
```

Environment Variables

Variable	Default	Purpose
PORT	3000	TCP port the HTTP server listens on. Must match the Replit/Docker port assignment.
LOG_PATH	"memory"	Audit log path. "memory" != in-process AppendOnlyLogService. Any other value != FileBackedAppendOnlyLogService at that path.
CONFIG_PATH	<cwd>/cerbaseal.config.json	Explicit path to the config file. Default: resolved from process.cwd().
POLICY_PATH	<cwd>/cerbaseal.policy.json	Explicit path to the policy file. Default: resolved from process.cwd().

10

Deployment Architecture

10.1 Path A — Docker Compose (Recommended)

Docker Compose is the recommended deployment path for partners delivering to client environments. It requires Docker and Docker Compose on the target machine — no Node.js, pnpm, or TypeScript toolchain is needed. The entire enforcement stack runs inside a self-contained container.

Property	Detail
Prerequisites	Docker "e 20, Docker Compose "e 2.0. No Node.js or pnpm required on host.
Build context	Repository root. The Dockerfile copies root src/ (cerbaseal-review) + deployment-starter/.
Base image	node:22-alpine (builder and runtime stages)
Build stages	Stage 1 (builder): install pnpm@9, pnpm install, resolve file:../ symlink to concrete files. Stage 2 (runtime): copy resolved deployment-starter + config files, expose port.
Runtime command	deployment-starter/node_modules/.bin/tsx deployment-starter/index.ts (from WORKDIR /app)
Audit log volume	./data:/app/data — audit.jsonl persists across container restarts on the host filesystem
Health check	wget -qO- http://localhost:3000/health — 30s interval, 5s timeout, 3 retries, 10s start period
Config override	Mount a custom cerbaseal.policy.json: -v ./my-policy.json:/app/cerbaseal.policy.json
Restart policy	restart: unless-stopped — container recovers from crashes automatically

Docker Compose quick start

```
# Quick start – from deployment-starter/ directory
cp .env.template .env           # copy template
# Edit .env: set CERBASEAL_ENV=production
docker compose up               # first run builds image (3–5 min)

# Confirm healthy
curl http://localhost:3000/health
# !' { "status": "ok", "gateReady": true, "auditChainValid": true, "uptime": 1.4 }
```

10.2 Path B — Node.js Direct

The Node.js direct path is used when the client environment has Node.js 18+ and pnpm available, or when partners need full development access to the source code during pilot configuration.

Step	Command	Expected Output
1 — Install	pnpm install	node_modules populated; pnpm-lock.yaml unchanged
2 — Test	pnpm test	391 tests passing (17 test files)
3 — Repo	pnpm audit:repo	16/16 checks passing
4 — Setup	pnpm setup	cerbaseal.config.json and cerbaseal.policy.json written
5 — Verify	tsx deployment-starter/verify.ts	9/9 assertions passing across 3 scenarios
6 — Export	pnpm export:proof	cerbaseal-proof.json written with checksum
7 — Verify	pnpm verify:proof	Checksum match confirmed
8 — Start server	tsx deployment-starter/index.ts	Server listening on PORT (default 3000)

Dependency Model

deployment-starter/package.json declares cerbaseal-review as a file: dependency pointing to the repository root ("cerbaseal-review": "file:../"). Running pnpm install in deployment-starter/ creates a node_modules/cerbaseal-review symlink to the repo root. When CerbaSeal-Core is published to npm, the reference changes to the npm package name — no other code changes required.

When publishing to npm in v0.2.0+, clients change one line: "cerbaseal-review": "file:../" ! "cerbaseal-review": "^0.2.0". All import paths remain identical.

11

Test Coverage

The CerberaSeal-Core test suite covers the enforcement gate, audit chain, evidence layer, configuration system, integration scenarios, adversarial integrity, and regression snapshots. All tests are run with Vitest. The test suite is a first-class deliverable — partners are required to run it and confirm 100% passing before any client deployment.

Test File	Category	Tests	What It Covers
<code>execution-gate-service.test.ts</code>	Unit	19	Core gate evaluation: all 12 invariants, policy rules, decision states, <code>GateResult</code> structure
<code>adversarial-integrity.test.ts</code>	Security	66	Tampering, bypass attempts, hand-constructed <code>GateResults</code> , invariant circumvention
<code>security/contextual-boundary.test.ts</code>	Security	25	Authority class boundaries, cross-context isolation, role escalation attempts
<code>security/fail-closed.test.ts</code>	Security	2	Gate fails closed (REJECT) on any schema error or unknown input
<code>security/non-forgery.test.ts</code>	Security	2	Audit chain hash non-forgery — tampered entries detected by <code>verifyChain()</code>
<code>security/misuse-scenarios.test.ts</code>	Security	34	Real-world misuse patterns: AI self-auth, stale controls, unsigned approvals
<code>audit-evidence-export.test.ts</code>	Integration	6	Evidence bundle assembly, export manifest, <code>ExportManifest</code> structure
<code>persistent-audit-log.test.ts</code>	Integration	12	<code>FileBackedAppendOnlyLogService</code> : crash recovery, chain persistence, verification
<code>diagnostic-report-service.test.ts</code>	Integration	5	<code>DiagnosticReport</code> assembly, severity classification, replay consistency
<code>integration/full-flow.test.ts</code>	Integration	1	End-to-end: request !' gate !' audit !' evidence !' export !' verify
<code>integration/browser-demo-routes.test.ts</code>	Integration	28	Browser demo HTTP server routes: all API endpoints and portal pages
<code>integration/review-portal-routes.test.ts</code>	Integration	110	Review portal routes: summary data, evidence, replay, diagnostic APIs
<code>integration/external-signal-examples.test.ts</code>	Integration	16	External actor integration patterns: multi-tenant, role mapping, chain verify
<code>integration/support-readiness.test.ts</code>	Integration	23	Support scenarios: operator action reports, system health checks, all 3 decision states
<code>integration/system-integration.test.ts</code>	Integration	1	Full system integration: gate + policy + audit + evidence working together
<code>config-policy-layer.test.ts</code>	Unit	—	Configuration loading, class extension, policy rule application
<code>snapshots/enforcement-loop.snapshot.test.ts</code>	Snapshot	41	Regression snapshots: specific request shapes produce identical <code>GateResult</code> outputs

All tests must pass (391 tests, 17 files) before any client deployment. The test suite is also run by `pnpm audit:repo` as check #1 of 16. A failing test suite blocks all further audit checks.

Repo Audit — 16 Automated Checks

`pnpm audit:repo` runs 16 checks against the repository state. These are separate from the test suite — they verify governance integrity properties of the repository itself, not just the source code behaviour.

Check #	What It Verifies
01	Test suite passes (391/391)
02	No prohibited imports in enforcement core (gate service cannot import from outside its layer)
03	Invariant count matches — exactly 12 invariants defined and referenced in tests
04	Every invariant code referenced in tests is present in the INVARIANTS constant
05	Reason codes referenced in tests are present in REASON_CODES constant
06	ExecutionGateService does not export internal state (WeakSet, _gateIssuedResults)
07	IAuditLogService interface is satisfied by both log implementations
08	FileBackedAppendOnlyLogService uses appendFileSync (not async write — crash safety)
09	DecisionEnvelope immutable field is literal type true in type definition
10	No direct test-to-implementation coupling (tests import via public API only)
11	cerbaseal.config.json parses without error and matches CerbaSealConfig schema
12	cerbaseal.policy.json parses without error
13	CerbaSealError carries invariant code, reason code, and finalState on every throw
14	No stale test-count references in documentation
15	Pilot documents exist and contain required sections
16	Export proof + verify proof round-trip produces identical checksums

12

Partner Delivery Framework

The Partner Delivery Kit (docs/partner-kit/) is the complete reference for a consulting partner delivering a CerbaSeal pilot to an enterprise client. It covers the full engagement lifecycle — from the first sales conversation to a completed pilot with a signed evidence package.

Partner Kit Documents

Document	File	Audience	Purpose
00 — Overview	00-OVERVIEW.md	All partners	Kit navigation, certification levels, supporting tools, escalation boundaries
01 — Sales Brief	01-sales-brief.md	Partner sales leads	Positioning, ICP, qualification criteria, commercial model, objection starters
02 — Technical Brief	02-technical-brief.md	Partner tech leads	7-layer architecture, invariant framework, evidence chain, integration patterns
03 — Deployment Guide	03-deployment-guide.md	Partner engineers	Step-by-step deployment: environment, setup wizard, audit, verification, evidence export
04 — 1-Day Pilot Guide	04-pilot-guide.md	Pilot leads	CerbaSeal Express Pilot: morning to evidence package in a single day
05 — Support Guide	05-support-guide.md	All partners	10 most common partner issues with root cause, diagnosis steps, and resolution
06 — Objection Handling	06-objection-handling.md	Partner sales leads	8 enterprise objections with persona-calibrated responses (CTO, CISO, CPO, CFO)
07 — Certification Framework	07-certification-framework.md	All partners	3 certification levels: competencies, practical exercises, pass/fail criteria

Certification Levels

Level	Name	What It Proves	Prerequisite	Minimum Before
Level 1	Deploy	Stand up CerbaSeal, run pnpm setup, pass pnpm audit:repo 16/16, run and interpret 3-scenario verification, export and verify a proof snapshot	None	Any client deployment
Level 2	Configure	Author a complete cerbaseal.policy.json for a new client workflow without assistance. All approval chains, actor mappings, action policies correct. Scenario verification passes	Level 1	Unsupervised pilot delivery
Level 3	Lead Pilots	Run a full pilot engagement independently from kickoff to signed evidence package. Deliver all 7 audit-kit documents to client. Resolve support issues without escalation.	Level 2 + one completed client delivery	Leading an engagement solo

Engagement Lifecycle

Pre-sales: Sales Brief qualification. ICP scoring. Technical discovery — 1 meeting with client CTO/CISO.

Scoping: Workflow mapping: identify the governed workflow, AI actor, human approver roles, action classes.

Deployment: Path A (Docker) or Path B (Node.js). 16/16 audit checks must pass before proceeding.

Configuration: Author cerbaseal.policy.json. Approval chains, actor mappings, action policies per client workflow.

Verification: Run 3 verification scenarios. All 9 assertions must pass. Review scenario outputs with client.

Pilot run: Client submits real or representative requests through the gate. Evidence is captured automatically.

Evidence close: Run `pnpm export:proof`. Verify with `pnpm verify:proof`. Deliver `cerbaseal-proof.json` to client.

Handover: Partner trains client operations team (Admin Guide in `docs/client-adoption/training/`).

What Requires Lamont Labs Tier 3 Escalation

Level 2+ partners are expected to resolve 80%+ of issues independently using the Partner Delivery Kit. The following items require direct Lamont Labs involvement:

- New core enforcement invariants — adding a new hard rule requires TypeScript changes and new tests
 - New proposal source kinds beyond "ai" and "deterministic_rule"
 - PKI-backed cryptographic signing infrastructure (HMAC is self-service; PKI requires implementation work)
 - Third-party security review coordination — not yet completed for v0.1.0
 - Windows deployment — not tested in v0.1.0; support posture is unclear
-

13

Security Properties & Threat Model

Core Security Properties

Property	Mechanism	Verification
No execution without gate evaluation	Every release authorisation is issued only by <code>ExecutionGateService.evaluate()</code> . The <code>WeakSet</code> registry enforces this at the <code>EvidenceBundleService</code>	INV-06; <code>assertIsGateIssued()</code> ; <code>misuse-scenarios.test.ts</code>
AI cannot self-authorise	<code>actorAuthorityClass "ai"</code> is structurally blocked from being the approving authority. INV-05 is checked on every request, unconditionally.	INV-05; <code>AI_CANNOT_AUTHORIZE</code> reason code; <code>adversarial-integrity.test.ts</code>
Tamper-evident audit log	SHA-256 forward chaining — each entry hash includes the previous entry hash. Any modification to a historical entry breaks the chain. <code>verifyChain()</code>	<code>FileBackedAppendOnlyLogService</code> ; <code>security/non-forgery.test.ts</code>
Immutable decision envelopes	<code>DecisionEnvelope.immutable</code> is a literal type <code>true</code> . TypeScript and runtime enforcement prevent mutation. The <code>WeakSet</code> registration ties the envelope	INV-07; <code>IMMUTABLE_DECISION_ENVELOPE</code>
Prohibited use unconditional block	<code>prohibitedUse: true</code> triggers REJECT at INV-10 before any policy rules are evaluated. No policy configuration can override a prohibited use block.	INV-10; <code>PROHIBITED_USE</code> reason code
Fail-closed on unknown input	Any unrecognised action class throws a <code>CerbaSealError</code> with REJECT. Any schema violation produces REJECT. The gate never silently permits	INV-11; <code>security/fail-closed.test.ts</code>
Policy cannot relax invariants	The policy layer is evaluated only after all 12 invariants pass. Policy rules can add HOLD or REJECT outcomes; they have no mechanism to	Gate evaluation order; <code>adversarial-integrity.test.ts</code>
No network dependency at evaluation	<code>gate.evaluate()</code> is a pure, synchronous function. It makes no network calls, reads no files, and has no side effects. Availability is not a gate availability concern.	<code>ExecutionGateService</code> source; <code>execution-gate-service.test.ts</code>

Threat Model — In Scope for v0.1.0

Threat	Attacker Goal	Mitigation
AI self-authorisation	AI system attempts to approve its own proposal	INV-05 blocks any <code>actorAuthorityClass "ai"</code> from being the approver. Structural — not policy-bypassable.
Approval forgery	Caller constructs a plausible-looking <code>ApprovalArtifact</code>	<code>immutableSignature</code> must be present (<code>APPROVAL_SIGNATURE_MISSING</code> check). Authority class must be in the approval chain. <code>priviledgedAuthSatisfied</code> must be <code>true</code>
Audit log tampering	Attacker modifies historical log entries to hide an event	SHA-256 forward chain detects any modification to any entry. <code>verifyChain()</code> is called on <code>GET /health</code> .
Gate bypass via evidence layer	Caller hand-constructs a <code>GateResult</code> and skips <code>gate.evaluate()</code>	<code>assertIsGateIssued()</code> in <code>EvidenceBundleService</code> rejects any <code>GateResult</code> not in the <code>WeakSet</code> .
Policy relaxation	Partner misconfigures policy to exempt a workflow from invariants	Policy layer cannot affect invariant evaluation. Invariants are evaluated before policy.
Stale control exploitation	Caller submits a sensitive request with outdated controls	INV-08 checks <code>controlStatus.stale</code> and <code>controlStatus.criticalControlsValid</code> for sensitive requests.
Prohibited use submission	Caller submits a prohibited use request with all other fields valid	INV-10 checks <code>prohibitedUse</code> first in the invariant sequence. REJECT is unconditional.

Out of Scope for v0.1.0

- PKI-backed cryptographic signing of DecisionEnvelope objects (HMAC available; PKI planned for v0.2.0)
- Third-party penetration testing or security audit (planned for v0.2.0)
- Network-level access control for the HTTP API (operator responsibility)
- Authentication/authorisation of API callers to the HTTP server (operator responsibility)
- Denial-of-service resilience (operator responsibility — rate limiting, WAF)

Security review status: CerbaSeal-Core v0.1.0 has not yet undergone a formal third-party security review. v0.2.0 includes a scheduled independent review as a milestone gate. Partners should communicate this status to clients during the pilot kickoff.

CerbaSeal-Core

v0.1.0 — System Architecture & Technical Reference

This document is the authoritative technical reference for CerbaSeal-Core.
It is issued under the Lamont Labs Partner Confidentiality Agreement.
Redistribution outside the authorised partner organisation is prohibited.

